

Universidade do Minho

Programação Orientada a Objetos

Desenvolvimento da aplicação DomusControl

Trabalho Prático – Grupo

Programação Orientada a Objetos

Membros do grupo:
aiai
aiai

Francisco Duarte Fernandes Silva – a100752

Universidade do Minho
2026
Licenciatura em Engenharia Informática

Índice

1 | Introdução 3
1.1 | Enquadramento do Problema 3
1.2 | O Paradigma da Programação Orientada aos Objetos 3
2 | Arquitetura de Classes e Decisões de Design 4
2.1 | Padrão de Arquitetura 4
2.2 | Modelação de Entidades Base 4
2.3 | Hierarquia de Dispositivos 4
3 | Gestão do Tempo, Automações e Cenários 6
3.1 | Simulação do Tempo 6
3.2 | Automações e Escalonamentos 6
3.3 | Cenários 6
4 | Estruturas de Dados e Estatísticas 7
5 | Geração Inteligente de Automações 8
6 | Persistência de Dados 9
7 | Diagrama de Classes 10
8 | Descrição da Aplicação 11
9 | Conclusões e Dificuldades Ultrapassadas 16

1 | Introdução

1.1 | Enquadramento do Problema

É visível na atualidade a passagem do conceito de "casas inteligentes" da ficção científica para a realidade. A existência de plataformas de gestão, como o *Home Assistant*, verdadeiramente revolucionou a organização doméstica, permitindo gerir de forma centralizada todos os equipamentos elétricos.

Com isto em mente, foi desenvolvido o DomusControl, uma aplicação de automação semelhante às anteriormente referidas, com as suas funcionalidades típicas, criada também com o objetivo de cumprir o paradigma da Programação Orientada aos Objetos.

1.2 | O Paradigma da Programação Orientada aos Objetos

A **Programação Orientada aos Objetos** é um paradigma de engenharia de software focado na modelação de sistemas através de **entidades discretas**, designadas por **objetos**, que agregam estado (**atributos**) e comportamento (**métodos**). Os seus princípios basilares assentam no **encapsulamento** (proteção do estado interno contra manipulações indevidas), na **abstração** (exposição apenas das interfaces e características essenciais ao exterior), na **herança** (reaproveitamento, especialização e hierarquização de código) e no **polimorfismo** (capacidade de tratar objetos de diferentes tipos sob uma mesma abstração).

No âmbito deste projeto, a implementação estrita destes princípios revelou-se a base fundamental para garantir a extensibilidade, segurança e robustez da aplicação. Em vez de focar o desenvolvimento numa resolução sequencial imperativa de rotinas, o desenho do sistema orientou-se pela transposição rigorosa do ambiente físico para uma arquitetura computacional modular, em que cada decisão arquitetónica procurou respeitar e validar estes quatro pilares teóricos.

Esta escolha de aproximação é fulcral aquando da programação orientada aos objetos.

2 | Arquitetura de Classes e Decisões de Design

O alicerce do projeto, tal como uma casa, parte da sua arquitetura. As classes foram pensadas com o objetivo de procurar operar com o máximo de coesão e o mínimo de acoplamento, respeitando boas práticas de modelação. A prioridade assentou no encapsulamento sistemático, assegurando que cada entidade atua como a exclusiva responsável pela manipulação do seu estado interno.

2.1 | Padrão de Arquitetura

Os requisitos impostos pelo paradigma ao design começam já na taxativa separação entre a interface de utilizador e o modelo de dados. A lógica de domínio (a classe central Domus) deve operar alheia à forma como a informação é apresentada. A utilização da classe TextUI e de menus dedicados para desenhar a aplicação (DomusApp) permitiu estabelecer uma barreira de abstração eficaz. Conseguiu-se isolar a camada de interação, abstendo o modelo de recorrer a métodos de entrada e saída diretos (como System.out.println), garantindo a pureza e portabilidade do código de domínio.

2.2 | Modelação de Entidades Base

Para modelar a disposição arquitetónica de uma habitação, recorreu-se aos conceitos de **Ocultação de Informação** (*Information Hiding*) e a **relações estritas de Composição**. O princípio do encapsulamento exige a proteção cabal dos atributos privados contra adulterações externas não controladas.

Para cumprir esta regra, aplicou-se exaustivamente a técnica de **cópias defensivas** (*defensive copying*).

Ao solicitar à entidade **Casa** a estrutura das suas divisões ou dispositivos, o sistema procede à instanciação de cópias idênticas em vez de expor as referências de memória originais.

Paralelamente, a alteração de estados foi estruturada segundo uma hierarquia de delegação de mensagens: o motor central delega a ação de gestão à Casa, que consequentemente a delega à Divisao, onde a operação final ocorre de forma encapsulada. Esta metodologia salvaguarda inequivocamente a integridade estrutural das entidades do sistema.

2.3 | Hierarquia de Dispositivos

O requisito de viabilizar a futura introdução de novos tipos de equipamentos sem implicar a reescrita de funções de controlo ditou o uso intensivo da Herança e do Polimorfismo (Dynamic Binding).

Concebeu-se a **superclasse abstrata Dispositivo**, dotada dos atributos e assinaturas metodológicas transversais a qualquer equipamento, originando especializações

concretas sob a forma das subclasses Lampada e Termostato. A invocação de comandos, como a rotina ligarOuDesligar, foi desenhada para operar na base da abstração superior. Assim, garante-se a uniformidade do tratamento, transferindo para a Máquina Virtual de Java (JVM) a responsabilidade de executar a lógica correspondente ao tipo exato do objeto instanciado, eliminando pesadas e contraproducentes sucessões condicionais (if-else ou o antipadrão instanceof).

3 | Gestão do Tempo, Automações e Cenários

3.1 | Simulação do Tempo

A representação temporal assumiu um papel indispensável para viabilizar as rotinas de escalonamento.

Optou-se pela **centralização de um relógio global** do tipo nativo *LocalTime* no seio da classe Domus. Por definição da linguagem, a classe *LocalTime* é estruturalmente imutável (padrão *Immutable Object*), ao bloquear modificações acidentais ou externas de referências.

O avanço da simulação propaga o novo estado temporal para as entidades subalternas, onde o **Polimorfismo** e a **Abstração** assumem novamente o controlo para avaliar condições pendentes de acionamento.

3.2 | Automações e Escalonamentos

Para manter a coesão com as boas práticas previstas, evitou-se alocar lógica imperativa excessiva no interior da classe Casa. Verificou-se a reidificação destes comportamentos transacionais, dando origem às classes representativas Escalonamento e Automacao.

A transmutação de verificações condicionais e de tempo em objetos isolados materializa o Princípio da Responsabilidade Única (SRP). A classe da Casa passa a atuar unicamente como gestora delegante, reservando a inteligência avaliativa temporal para o interior dos próprios objetos de automação.

3.3 | Cenários

A implementação dos **Cenários** adotou idêntico princípio de reificação. O modelo de domínio Cenario abstrai uma coleção coerente de objetos do tipo Acao. Atividades abrangentes, como o fechar sistemático da habitação, concretizam-se através da iteração encapsulada e execução polimórfica sobre os dispositivos visados.

As rotinas são acauteladas com tratamento apropriado de exceções de domínio (e.g. tentativa de ação sobre dispositivo inexistente), o que garante o cumprimento do princípio de robustez do software ditado pelas normativas do tratamento de *Exceptions* em Java e POO.

4 | Estruturas de Dados e Estatísticas

O alojamento, acesso e cruzamento analítico de grandes conjuntos de dados, indispensável ao cálculo de estatísticas e consumos elétricos, requereu a eleição cuidada de **Estruturas de Dados** inseridas na *Collections Framework* da linguagem (Map, List).

A elevação da eficácia analítica e da aderência ao paradigma processou-se pela **substituição de iterações imperativas 'tradicionais' pela agilidade funcional da Streams API**.

O cálculo de somatórios e a extração de objetos de consumo máximo foram solucionados com **operações declarativas**, recorrendo sintaticamente a **expressões Lambda e *Method References*** (ex: Dispositivo::getConsumoPorHora). Acresce que a definição dos algoritmos de ordenação (*Comparators*) passou a ser alojada em mapas não modificáveis (*Collections.unmodifiableMap(…)*), providenciando proteção absoluta contra quebras de encapsulamento no plano das constantes estáticas.

5 | Geração Inteligente de Automações

6 | Persistência de Dados

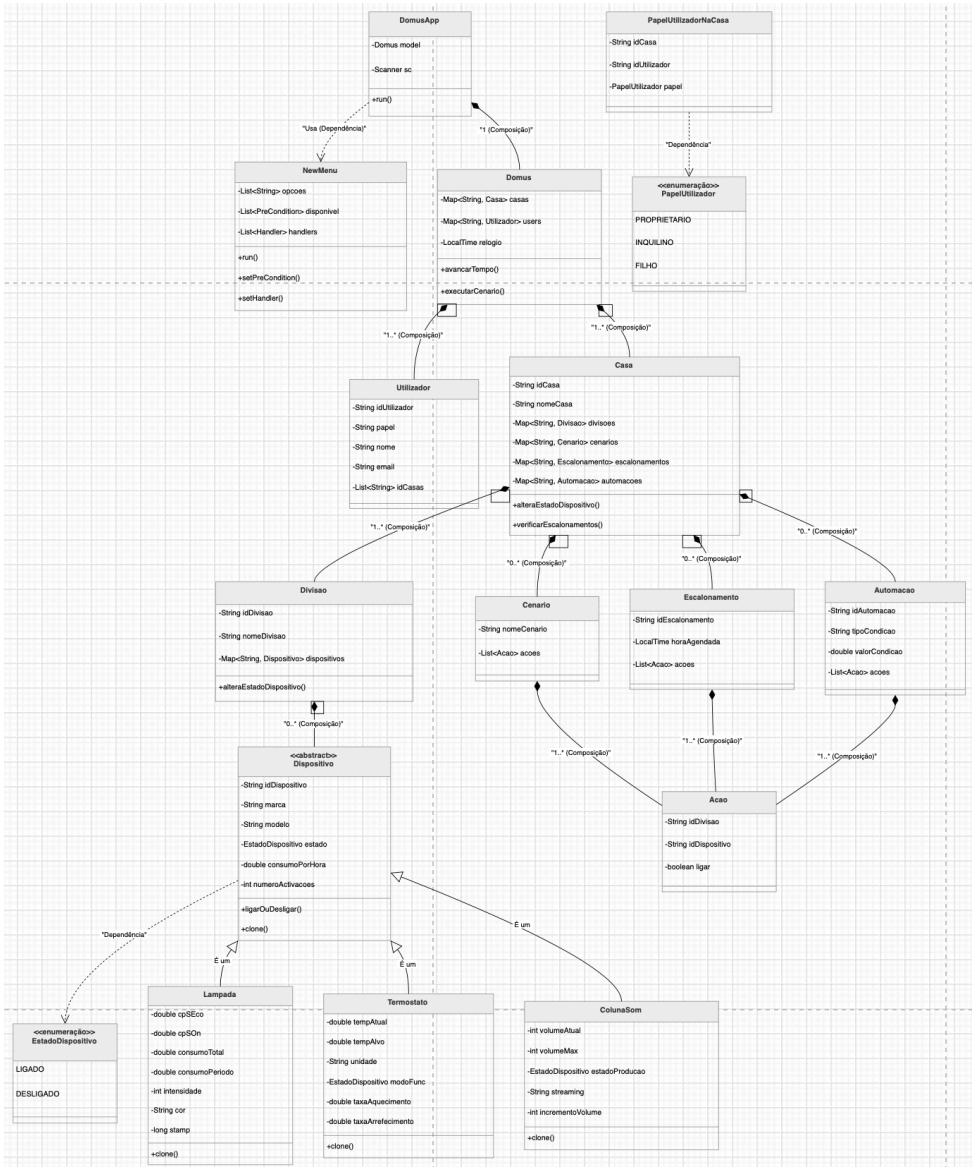
O mecanismo para assegurar a **persistência de toda a topologia de dispositivos e registos temporais** utilizou, na íntegra, a funcionalidade de **Serialização de Objetos** em Java *(Serializable)*.

Do **ponto de vista** puramente **orientado a objetos**, a **serialização** estabelece-se como o **método normativo** predileto. Caracteriza-se pela sua capacidade singular de traduzir e restaurar fielmente o grafo completo das referências alojadas na Memória RAM sem interrupções de lógica.

Excluiu-se deliberadamente a opção de leitura e escrita de dados por via de ficheiros textuais convencionais (txt/csv), em virtude de essa alternativa exigir a construção de analisadores linguísticos (*parsers*) que invariavelmente requereriam a exposição inadequada das propriedades protegidas pelo encapsulamento.

7 | Diagrama de Classes

A idealização da arquitetura e interdependência de pacotes que compõem o Domus-Control foi previamente fixada utilizando o Unified Modeling Language (UML), em anexo.



8 | Descrição da Aplicação

Consolidada a solidez das restrições arquitetónicas em todas as vertentes, a interação processou-se fluidamente em modo terminal de texto. (Ilustrar esta secção através da inserção sequencial de capturas do terminal:)

Demonstração das opções de criação hierárquica e instaciação (Composição em prática).

```
 *** NewMenu ***
1 - Gestão de Casas
2 - Gestão de Utilizadores
3 - Estatísticas
4 - Simulação e Cenários
5 - Guardar estado
6 - Carregar estado
0 - Sair
Opção: 1

 *** NewMenu ***
1 - Adicionar Casa
2 - Listar Casas
3 - Remover Casa
4 - Gerir Divisões de uma Casa
0 - Sair
Opção: 1
Id da casa: 12
Nome da casa: casapovoa
Casa adicionada com sucesso.

 *** NewMenu ***
1 - Adicionar Casa
2 - Listar Casas
3 - Remover Casa
4 - Gerir Divisões de uma Casa
0 - Sair
Opção: 4
Id da casa: 12

 *** NewMenu ***
1 - Adicionar Divisão
2 - Listar Divisões
3 - Remover Divisão
4 - Gerir Dispositivos de uma Divisão
0 - Sair
Opção: 1
Id da divisão: 12
Nome da divisão: sala
Divisão adicionada com sucesso.
```

Prova das capacidades Polimórficas instanciando dispositivos de naturezas diferentes na mesma Coleção de uma Divisão.

```
 *** NewMenu ***
1 - Adicionar Casa
2 - Listar Casas
3 - Remover Casa
4 - Gerir Divisões de uma Casa
0 - Sair
Opção: 4
Id da casa: 12

 *** NewMenu ***
1 - Adicionar Divisão
2 - Listar Divisões
3 - Remover Divisão
4 - Gerir Dispositivos de uma Divisão
0 - Sair
Opção: 1
Id da divisão: 12
Nome da divisão: sala
Divisão adicionada com sucesso.

 *** NewMenu ***
1 - Adicionar Divisão
2 - Listar Divisões
3 - Remover Divisão
4 - Gerir Dispositivos de uma Divisão
0 - Sair
Opção: 4
Id da divisão: 12
Id do dispositivo: 12
Deseja 'Ligar' ou 'Desligar' + o Dispositivo:

 *** NewMenu ***
1 - Adicionar Dispositivo
2 - Listar Dispositivos
3 - Remover Dispositivo
4 - Operar Dispositivo
0 - Sair
Opção: 1

 *** NewMenu ***
1 - Lâmpada
2 - Termostato
0 - Sair
Opção: 1
Id: 123
Marca: rowenta
Modelo: 2
```

```
Consumo por hora (Wh): 122
Lâmpada adicionada com sucesso.

 *** NewMenu ***
1 - Lâmpada
2 - Termostato
0 - Sair
Opção: 0

 *** NewMenu ***
1 - Adicionar Dispositivo
2 - Listar Dispositivos
3 - Remover Dispositivo
4 - Operar Dispositivo
0 - Sair
Opção: 2
Id: 123 | Marca: rowenta | Modelo: 2 | Estado: OFF


 *** NewMenu ***
1 - Adicionar Dispositivo
2 - Listar Dispositivos
3 - Remover Dispositivo
4 - Operar Dispositivo
0 - Sair
Opção: 1

 *** NewMenu ***
1 - Lâmpada
2 - Termostato
0 - Sair
Opção: 2
Id: 134
Marca: chin
Modelo: ade
Consumo por hora (Wh): 12
Temperatura atual: 30
Temperatura alvo: 25
Termostato adicionado com sucesso.

 *** NewMenu ***
1 - Lâmpada
2 - Termostato
0 - Sair
Opção: 0

 *** NewMenu ***
1 - Adicionar Dispositivo
2 - Listar Dispositivos
3 - Remover Dispositivo
```

```
4 - Operar Dispositivo
0 - Sair
Opção: 2
Id: 123 | Marca: rowenta | Modelo: 2 | Estado: OFF
Id: 134 | Marca: chin | Modelo: ade | Estado: OFF
```

O acionamento integral e sucessivo das ações de um Cenário, atestando a integridade das ordens de delegação implementadas.

```
 *** NewMenu ***
1 - Gestão de Casas
2 - Gestão de Utilizadores
3 - Estatísticas
4 - Simulação e Cenários
5 - Guardar estado
6 - Carregar estado
0 - Sair
Opção: 4
Hora Atual do Sistema: 00:00

 *** NewMenu ***
1 - Avançar no Tempo
2 - Executar Cenário Manual
3 - Carregar Dados de Teste Ponto 8.3 (Cria Cenários e Escalonamentos base)
0 - Sair
Opção: 3
Id da Casa para injetar testes: 12
Cenário 'Sair de Casa' e Escalonamento 'Acordar' às 08:00 criados na casa 12

 *** NewMenu ***
1 - Avançar no Tempo
2 - Executar Cenário Manual
3 - Carregar Dados de Teste Ponto 8.3 (Cria Cenários e Escalonamentos base)
0 - Sair
Opção: 2
Id da Casa: 12
Nome do Cenário: Sair de Casa
Cenário executado.
```

O salto temporal e o subsequente despoletar dos Escalonamentos reificados no sistema global.

```
 *** NewMenu ***
1 - Gestão de Casas
2 - Gestão de Utilizadores
3 - Estatísticas
4 - Simulação e Cenários
5 - Guardar estado
6 - Carregar estado
```

```
0 - Sair
Opção: 4
Hora Atual do Sistema: 00:00

 *** NewMenu ***
1 - Avançar no Tempo
2 - Executar Cenário Manual
3 - Carregar Dados de Teste Ponto 8.3 (Cria Cenários e Escalonamentos base)
0 - Sair
Opção: 1
Quantas horas avançar: 0
Quantos minutos avançar: 45
Tempo avançado com sucesso. Nova hora: 00:45
```

9 | Conclusões e Dificuldades Ultrapassadas

O percurso de desenvolvimento desta aplicação materializou a verificação empírica de que a obediência às exigências do paradigma Orientado a Objetos demanda uma estruturação intensiva e prévia, ao impôr requisitos que, a princípio, dificultaram o planeamento e resolução. A estrita obrigação de suprimir acessos diretos em favor de cópias defensivas, as restrições impostas pelos modificadores de acesso, e a necessidade arquitetónica de desenhar cascatas de delegação de mensagens, causaram num estágio inicial uma latência substancial ao volume de código gerado.

Todavia, a real valia destas fundações foi plenamente atestada nos momentos de integração final do projeto. O isolamento celular que encapsula cada secção evitou ramificações caóticas e corrupção não rastreável de estados em memória. Fica demonstrado, em suma, que **a severidade da POO não atua como uma barreira ao código**, sendo sim a derradeira e **mais eficaz infraestrutura para a conceção de software altamente seguro**, flexível à mudança e arquitetonicamente perene.